Task 2: Math Operations Using SVC Number

```
/**

*****************************************************************************
*******
 * @file           : main.c
 * @author         : Sagar More
 * @brief          : svc math operations

*****************************************************************************
*******
 */

#include <stdint.h>
#include <stdio.h>

/* Function to invoke SVC */
static inline int svc_call(int a, int b, int svc_no)
{
    register int r0 __asm("r0") = a;
    register int r1 __asm("r1") = b;
    register int r12 __asm("r12") = svc_no;

    __asm volatile (
        "svc #0"
        : "+r"(r0)
        : "r"(r1), "r"(r12)
        : "memory"
    );

    return r0;
}

/* Thread mode code */
int main(void)
{
    int a = 20;
    int b = 5;
    int result;

    result = svc_call(a, b, 36);
    printf("Addition: %d\n", result);

    result = svc_call(a, b, 37);
    printf("Subtraction: %d\n", result);

    result = svc_call(a, b, 38);
    printf("Multiplication: %d\n", result);

    result = svc_call(a, b, 39);
    printf("Division: %d\n", result);

    while (1);
}
```

```
/* Naked SVC handler */
__attribute__((naked)) void SVC_Handler(void)
{
    __asm volatile (
        "TST lr, #4        \n"
        "ITE EQ            \n"
        "MRSEQ r0, MSP     \n"
        "MRSNE r0, PSP     \n"
        "B SVC_Handler_C   \n"
    );
}

/* C SVC handler */
void SVC_Handler_C(uint32_t *pStackFrame)
{
    uint32_t op1 = pStackFrame[0];   // R0
    uint32_t op2 = pStackFrame[1];   // R1
    uint32_t result = 0;

    /* Extract SVC number */
    uint8_t *pc = (uint8_t *)pStackFrame[6];
    uint8_t svc_number = pc[-2];

    switch (svc_number)
    {
        case 36:
            result = op1 + op2;
            break;

        case 37:
            result = op1 - op2;
            break;

        case 38:
            result = op1 * op2;
            break;

        case 39:
            result = (op2 != 0) ? (op1 / op2) : 0;
            break;

        default:
            result = 0;
            break;
    }

    /* Return value via stacked R0 */
    pStackFrame[0] = result;
}
```